Problems on Trees

DAY 179
05 / Jan / 2026

Problem 19 : Give an algorithm for finding the level that has the maximum Sum in the Binary Tree.

Solution : The logic is very similar to finding the number of levels. The only change is, we need to keep track of the Sums as well.

Algorithm

1. if root == null return -1
2. Create an Empty Queue Q
3. Enque root into Q
4. Initialize
   - level = 0
   - maxSum = Integer.MIN_VALUE
   - maxLevel = 0
5. While Q is not Empty
   - Increment level By 1
   - Set level Size = Q.size()
   - Set currentSum = 0
   - Repeat level Size times
     - Dequeue a node curr
     - Add curr.data to currentSum
     - Enqueue left Child if Exists
     - Enque right Child if Exists
   - if currentSum > maxSum
     - Update maxSum = currentSum
     - Update maxLevel = level
6. Return Max Level (or maxSum, if asked)

Code

```java
public static int maxSumLevel(TreeNode root) {
    if (root == null)
        return -1;

    Queue<TreeNode> q = new LinkedList<>();
    q.add(root);

    int level = 0;
    int maxLevel = 0;
    int maxSum = Integer.MIN_VALUE;

    while (!q.isEmpty()) {
        level++;
        int levelSize = q.size();
        int currentSum = 0;

        for (int i = 0; i < levelSize; i++) {
            TreeNode cur = q.poll();
            currentSum += cur.data;

            if (cur.left != null)
                q.add(cur.left);
            if (cur.right != null)
                q.add(cur.right);
        }

        if (currentSum > maxSum) {
            maxSum = currentSum;
            maxLevel = level;
        }
    }
    return maxLevel;
}
```

Problem 20 : Given a Binary Tree, print out its root-to-leaf paths.

Solution : Traverse the tree. Maintain a path array / list
When leaf node is reached → print the stored path
Use Backtracking to remove nodes when Returning

Algorithm

1. if root == null, return
2. Add root.data to path[pathLen]
3. Increment pathLen
4. If root is a leaf Node
   - print elements from path[0] to path[pathLen-1]
5. Else
   - Recursively call for left Subtree
   - Recursively call for Right Subtree
6. Backtracking Happens when function Returns.

Code

```
public static void printPaths(TreeNode root) {
    StringBuilder path = new StringBuilder();
    printPathsUtil(root, path);
}

private static void printPathsUtil(TreeNode root, StringBuilder path) {
    if (root == null)
        return;

    int len = path.length(); // save length for backtracking

    path.append(root.data);

    // if leaf node, print path
    if (root.left == null && root.right == null) {
        System.out.println(path.toString());
    } else {
        path.append(" -> ");
        printPathsUtil(root.left, path);
        printPathsUtil(root.right, path);
    }

    // backtrack
    path.setLength(len);
}
```

Problem 21 Give an Algorithm for checking the Existence of path with given Sum. That means, given a Sum, check Wheather there Exists a path from root to any of the nodes

Solution: For this Problem, the strategy is: Substract the node value from the Sum Before calling its children recursively, and check to see if the Sum is 0 when we run out of tree.

Algorithm

1. if root == null return false
2. Substract root.data from Sum
3. if Sum == 0, return true
4. Recurssively check
   - left Subtree with updated Sum
   - Right Subtree with updated Sum
5. if Eigther returns true then returns true.
6. otherwise return false

Code

```
public static boolean hasPathSum(TreeNode root, int sum) {
    if (root == null)
        return false;

    // subtract current node value
    sum -= root.data;

    // if sum becomes zero at any node
    if (sum == 0)
        return true;

    // check left or right subtree
    return hasPathSum(root.left, sum) ||
           hasPathSum(root.right, sum);
}
```

Problem 22 Give an algorithm for finding the Sum of all Elements in Binary Tree.

Solution Recursively call left Subtree & right Subtree Sum and add their values to current nodes data.

Algorithm

1. if root == null return 0.
2. return root.data + TreeSum(root.left) + TreeSum(root.Right)

Code

```
public class SumOfTree {

    public static int treeSum(TreeNode root) {
        if (root == null)
            return 0;

        return root.data
             + treeSum(root.left)
             + treeSum(root.right);
    }
```

Problem 23 finding Sum problem non Recursive

Solution We can use level order traversal with Simple change. Every time after an Element from Queue, add the nodes data value to Sum Variable.

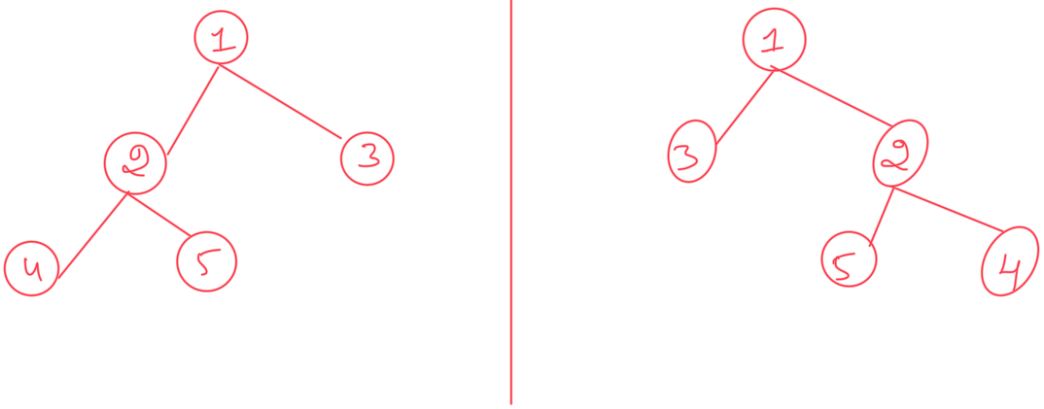
Algorithm

1. if root == null, return 0
2. Create an Empty Queue Q
3. Enqueue root into Q
4. Initialize Sum = 0
5. While Q is not Empty
   - Dequeue a node
   - add cur.data to Sum
   - if cur.left Exists Enqueue
   - if cur.Right Exists Enqueue
6. Return Sum

Code

```
public static int treeSum(TreeNode root) {
    if (root == null)
        return 0;

    Queue<TreeNode> q = new LinkedList<>();
    q.add(root);

    int sum = 0;

    while (!q.isEmpty()) {
        TreeNode cur = q.poll();
        sum += cur.data;

        if (cur.left != null)
            q.add(cur.left);
        if (cur.right != null)
            q.add(cur.right);
    }
    return sum;
}
```

Problem 24 Give an algorithm for converting a tree to its mirror. Mirror of a tree is another tree with left & right children of all non-leaf nodes interchanged.



Solution: Swaping left & Right children of Every Node

Algorithm

1. if root == null, return
2. Recursively convert left subtree into mirror
3. Recursively convert Right Subtree into mirror
4. Swap root.left and root.right

Code

```
public static void mirror(TreeNode root) {
    if (root == null)
        return;

    // mirror left and right subtrees
    mirror(root.left);
    mirror(root.right);

    // swap left and right
    TreeNode temp = root.left;
    root.left = root.right;
    root.right = temp;
}
```